

Insecure Data Storage & Sensitive Data Exposure

Introduction

Definitions

Data storage

It is the process of retaining digital information on electronic devices like mobile phones, including credentials and personal data.

Insecure data storage

It occurs when sensitive information is kept on a device in an unencrypted or improperly secured manner, such as in plain text or easily accessible locations.

Sensitive data exposure

Happens when confidential information is unintentionally revealed to unauthorized parties due to insecure transmission, poor encryption, or mishandling.

Vulnerabilities and Discovery Methods

- **Insecure Storage Identification**
 - Security experts examine source code and shared preference files, search for database paths, and look for temporary files created by the application.
- **Sensitive Data Exposure Analysis**
 - Methods include decompiling the APK to access underlying code and using scripts to search for hardcoded secrets or sensitive keywords.

- **Methods for android application (APK) include**
 - Examining the source code of the APK for any juicy keywords.
 - Attempt to decompile the APK to access its underlying code.
 - Use commands or scripts to search for specific keywords or patterns within the APK's file.

What is the impact of this issue?

- **Information Disclosure and Data Breaches**
 - Unauthorized access to personal, financial, and critical data stored on the device leads to severe breaches and privacy violations.
- **Identity Theft and Financial Loss**
 - Stolen data results in fraudulent transactions, unauthorized account access, and long-term harm to individuals' identities.
- **Reputational Damage**
 - Reputational loss for an application developer can cause huge loss for all other apps under the same developer.

Case Study

The Plaintext Preference Leak

A developer for a retail application implemented a "Remember Me" feature to enhance user experience. To save time, the application was designed to store session tokens and user profile information locally on the mobile device.

The Vulnerability (Insecure Data Storage)

The application stored sensitive session tokens and personal details within a shared preference XML file in plaintext. There was no encryption applied to the data at rest, making it accessible to any process or individual with access to the device's file system.

The Attack

An attacker utilized a compromised backup of the device or gained physical access to the unlocked phone. By navigating to the application's private data directory, they located the `user_session.xml` file and read the plaintext credentials and session tokens without needing any decryption keys.

Result

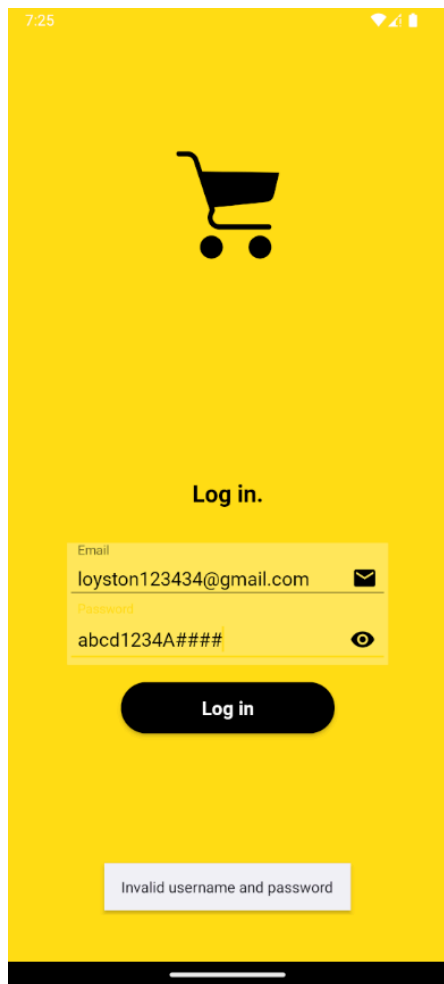
The attacker successfully hijacked the user's session and gained full control over the retail account, accessing stored payment methods and personal history. The incident resulted in significant financial theft and a complete erosion of customer trust in the application provider.

Practical (Insecure Shop)

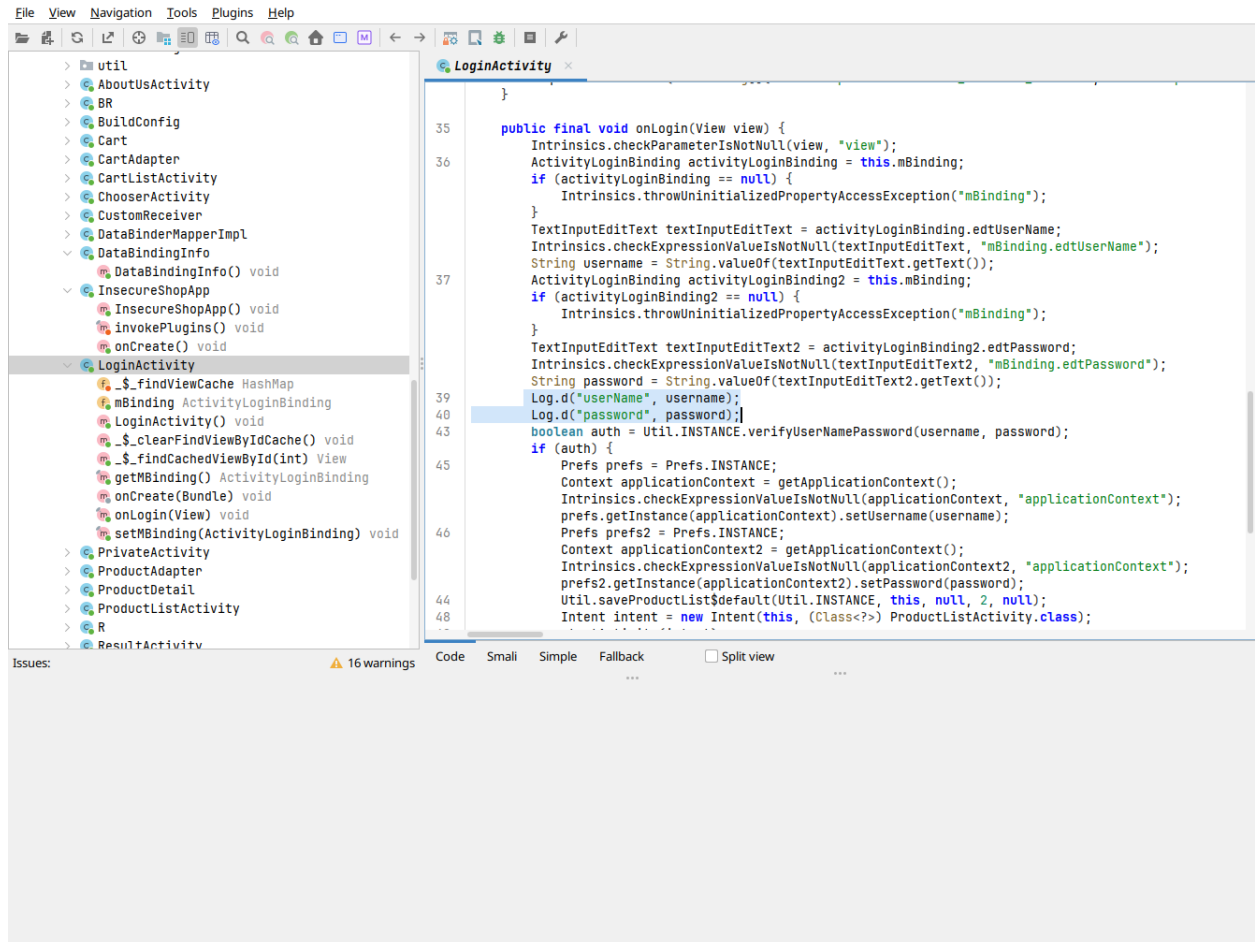
We first install InsecureShop.apk via adb install

```
> ~/Downloads via  via  via  via  via  v3.13.12 via  default via t
Z> adb install InsecureShop.apk
Performing Streamed Install
Success
```

Then, try to login into Insecure Shop



To see why the app rejects our login attempts, we check its source by decompiling the apk via jadx.



We find our first vulnerability in LoginActivity. The method onLogin is logging the username and password which can be accessed by running **logcat** command via adb.

```
> /tmp via
Z> adb shell
emu64xa:/ # logcat
```

When the user enters their username and password, the details are immediately logged which can be accessed by hackers.

```
ms count=2
05-15 19:20:50.552 4900 4915 D EGL_emulation: app_time_stats: avg=427.65ms min=283.2
ms count=3
05-15 19:20:50.710 4900 4900 D userName: loyston123434@gmail.com
05-15 19:20:50.710 4900 4900 D password: abcd1234A####
05-15 19:20:50.710 463 2929 D audioserver: logFgsApiBegin: FGS Logger Transaction f
05-15 19:20:50.710 622 1990 W AS.PlaybackActivityMon: No piid assigned for invalid/
id 15
05-15 19:20:50.734 622 1212 D CoreBackPreview: Window{2cbee9c u0 Toast}: Setting ba
BackInvokedCallbackInfo{mCallback=android.window.IOnBackInvokedCallback$Stub$Proxy@69a
y=0, mIsAnimationCallback=false}
05-15 19:20:50.757 463 584 D AudioFlinger: mixer(0x7182806ac8e0) throttle_end: the
```

Mitigations

- **Implement Strong Encryption**
 - Ensure all sensitive data stored on the device is encrypted using robust algorithms like AES-256.
- **Use Secure Storage Containers**
 - Leverage platform-specific secure storage like Android Keystore or iOS Keychain for managing keys and secrets.
- **Minimize Data Retention**
 - Avoid storing sensitive data on the device unless absolutely necessary, and implement secure deletion for temporary files.

Conclusion

Insecure data storage and sensitive data exposure remain critical threats in mobile application security, often stemming from inadequate encryption and mishandling of confidential information. These vulnerabilities can lead to devastating data breaches, financial loss, and severe reputational damage. To safeguard user information, developers must prioritize secure-by-design practices, implementing robust encryption and utilizing secure platform storage to maintain integrity and trust.

References

<https://github.com/hax0rgb/InsecureShop>

<https://owasp.org/www-project-mobile-top-10/2023-risks/m9-insecure-data-storage>